Daily Quant Digest: 2026-11-04

Foundations Restart, day 9. Three new steps today: how to **turn a tree around** and ask "given what I saw, which case was I in?" (Bayes' rule, and its fast odds form), how to measure **how much a bond moves** when yields move (duration and DV01, the numbers every rates desk lives by), and **recursion**, a function that solves a problem by handing a smaller copy of it to itself.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 9: Bayes' rule: counting people, the odds form	Easy-Medium
Derivatives	FR Lesson 9: duration and DV01	Easy-Medium
Algorithms & DS	FR Lesson 9: recursion and the call stack: fast power x^n	Medium
Music	Lynyrd Skynyrd, "Free Bird" (1973)	

1. Probability & Statistics: Bayes' Rule, or Turning the Tree Around

> **Where we are.** Lesson 6: news **moves the walls in**, and $P(A \mid B)$ is the share of the small room B taken by A. Lesson 8: to find $P(A)$, split the world into cases, multiply along each path of the tree, and add the paths. Yesterday's last line: of the 190 traders who lost money, 100 were on the FX desk, so $P(\text{FX} \mid \text{lost}) = 100/190 \approx 53\%$, even though only 20% of FX traders lost. Today we make that "turning around" into a tool.

1.1 The central picture: the airport beep

Every day, thousands of people walk through an airport metal detector. A tiny number carry something dangerous. The machine almost always beeps for them. But it also beeps for belts, keys and watches.

Now stand in the small room where everyone who **beeped** waits for a hand search. Look around. Who is in this room? Almost all of them are ordinary travellers with a belt buckle. A few are the people the machine was built to catch.

That room is the whole idea of Bayes' rule. A beep is **evidence**. It moves the walls in (Lesson 6): we stop looking at all travellers and look only at the ones who beeped. The question is: inside this new room, what share are real threats?

In the picture	In the maths
All travellers	The whole room, probability 1
Carries something dangerous	The hypothesis H
Share of travellers who carry something	The **prior** $P(H)$: what you believe before the evidence
The beep	The evidence E
Beep rate for carriers	$P(E \mid H)$, how well the test catches
Beep rate for ordinary travellers	$P(E \mid \text{not } H)$, the **false alarm** rate
The waiting room of beepers	The new, smaller room E
Share of carriers in that room	The **posterior** $P(H \mid E)$: what you believe after the evidence

Where the analogy stops: a real security officer also looks at the person, the bag and the X-ray. Each of those is **more evidence**, and Bayes tells us how to stack it, which is exactly section 1.5.

1.2 Count first: 1,000 traders and a surveillance alert

A bank's compliance system scans every trader's orders for **spoofing** (placing fake orders to push the price). Suppose:

- 2% of traders are actually spoofing.
- The alert fires for 90% of spoofers.
- The alert also fires for 5% of honest traders (they just trade in a way that looks similar).

The alert fires on Alex. How likely is it that Alex is spoofing? Most people guess "about 90%". Let us count.

Imagine **1,000 traders**:

	Traders	Alert fires	Alert silent
Spoofers (2%)	20	$20 \times 0.90 = 18$	2
Honest (98%)	980	$980 \times 0.05 = 49$	931
Total	**1,000**	**67**	**933**

Now walk into the "alert fires" room. It holds $18 + 49 = 67$ people. Only 18 of them are spoofers. So

$$P(\text{spoofer} \mid \text{alert}) = \frac{18}{67} \approx 26.9\%.$$

Not 90%. Not even close. Why? Because honest traders are so many (980) that even a small false alarm rate (5%) produces more alarms (49) than the real cases do (18). This is called **base rate neglect**: forgetting how rare the thing was to begin with. It is the single most common Bayes mistake, in interviews and in real life.

1.3 The formula, one step at a time

In words: the chance of the hypothesis given the evidence is "the evidence paths that go through the hypothesis" divided by "all the evidence paths".

$$P(H \mid E) = \frac{P(H)\,P(E \mid H)}{P(E)}, \qquad P(E) = P(H)\,P(E \mid H) + P(\text{not } H)\,P(E \mid \text{not } H).$$

What each symbol means. $P(H)$ is the prior (2%). $P(E \mid H)$ is how often the evidence shows up when H is true (90%). The bottom line, $P(E)$, is the total chance of seeing the evidence at all, and it is **exactly yesterday's law of total probability**: split into "spoofer" and "honest", multiply along each path, add.

Read aloud: "the probability of H given E equals the probability of H times the probability of E given H, divided by the total probability of E."

Why it is true, in two lines. By Lesson 6, $P(H \mid E) = P(H \cap E)/P(E)$: the share of room E taken by H. By the multiplication rule, $P(H \cap E) = P(H)\,P(E \mid H)$. Substitute. That is all Bayes' rule is: the definition of conditional probability, with the top written one way and the bottom expanded as a tree.

With numbers:

$$P(\text{spoofer} \mid \text{alert}) = \frac{0.02 \times 0.90}{0.02 \times 0.90 + 0.98 \times 0.05} = \frac{0.018}{0.018 + 0.049} = \frac{0.018}{0.067} \approx 0.269.$$

Same as the count: 18 out of 67. The table and the formula are the same calculation; the table just uses people instead of decimals.

1.4 The odds form: the fast way

Odds are a different way to write a probability: "for each one who is, how many are not". 20 spoofers and 980 honest traders is **odds of 20 to 980**, or 1 to 49, written $1:49$. To go back to a probability: $1/(1+49) = 2\%$.

Look at the "alert fires" column again. We had 18 spoofers to 49 honest. Where did 18 and 49 come from?

- Spoofers: 20×0.90.
- Honest: 980×0.05.

So the new odds are the old odds, with each side multiplied by its own alert rate:

$$\underbrace{\frac{18}{49}}_{\text{posterior odds}} = \underbrace{\frac{20}{980}}_{\text{prior odds}} \times \underbrace{\frac{0.90}{0.05}}_{\text{likelihood ratio}} = \frac{1}{49} \times 18.$$

In words: posterior odds = prior odds × likelihood ratio.

The likelihood ratio $P(E \mid H)/P(E \mid \text{not } H)$ measures how strongly the evidence points to H. Here it is $0.90/0.05 = 18$: an alert is 18 times more likely to come from a spoofer than from an honest trader. A ratio of 1 means the evidence is useless (both types produce it equally often). A ratio below 1 means the evidence points **away** from H.

Why it is true. Write Bayes' rule once for H and once for "not H", and divide one by the other. The awkward bottom line $P(E)$ is the same in both, so it **cancels**. That is the whole trick: odds let you skip the law of total probability.

Read aloud: "after seeing the evidence, the odds of H are the odds before, times how many times more likely the evidence is under H than under not-H."

1.5 Stacking evidence

Alex's case goes to a second, **independent** check (a human reviewer), which also catches 90% of spoofers and wrongly flags 5% of honest traders. It flags Alex too. "Independent" here means: once we know which type Alex is, the two checks do not influence each other (Lesson 7, applied inside each case).

With odds, each new piece of evidence is just one more multiplication:

$$\text{odds} = \frac{1}{49} \times 18 \times 18 = \frac{324}{49}, \qquad P = \frac{324}{324 + 49} = \frac{324}{373} \approx 86.9\%.$$

Count to check. Of the 18 flagged spoofers, 90% are flagged again: 16.2. Of the 49 flagged honest traders, 5% are flagged again: 2.45. Share of spoofers: $16.2/(16.2 + 2.45) = 0.869$. Same answer.

This is how a good trader thinks about information: start from a base rate, and let each new fact **multiply** your odds up or down. Not "jump to the answer the last fact suggests".

1.6 Common mistakes

- **Confusing $P(E \mid H)$ with $P(H \mid E)$.** "The alert catches 90% of spoofers" is not "90% of alerts are spoofers". In our numbers the second is only 27%.
- **Ignoring the base rate.** A great test for a rare thing still produces mostly false alarms. Always ask: how rare was it before?
- **Treating dependent evidence as independent.** If the reviewer only looks at the same order pattern the computer already used, the second flag adds almost nothing. Multiplying likelihood ratios only works when the pieces of evidence are independent given the case.

1.7 Self-check

1. 2% of trades have a booking error. A control flags 95% of bad trades and 10% of good ones. Count with 1,000 trades: what share of flagged trades are bad?
2. Using yesterday's three desks (FX 500 traders with 100 losers, rates 300 with 30, equities 200 with 60): given a trader lost money, what is the chance she is on equities?
3. Your odds that a stock rallies after earnings are $1:3$. A signal with likelihood ratio 6 fires. What are your new odds, and your new probability?

1.8 Interview question: is this client sharp?

(Market makers at Optiver, SIG and Citadel Securities think like this all day. The interviewer wants the base rate and the odds update.)

You make prices to clients. 20% of your clients are **sharp**: after a sharp client trades with you, the market moves against you 80% of the time. After a normal client trades, it moves against you 40% of the time. Assume each trade's outcome is independent once you know the client's type.

a. A new client trades once and the market moves against you. What is the probability she is sharp?

b. Instead, she trades three times and the market moves against you all three times. Now what?

c. How many "against" moves in a row before you are at least 90% sure she is sharp?

d. After the three against moves in (b), her fourth trade goes **your** way. What is the probability now?

1.9 Solution

(a) Count 1,000 clients. 200 sharp, 800 normal. After one trade, "against" happens for $200 \times 0.8 = 160$ sharp and $800 \times 0.4 = 320$ normal clients.

$$P(\text{sharp} \mid \text{against}) = \frac{160}{160 + 320} = \frac{160}{480} = \frac{1}{3}.$$

With odds: prior odds $200:800 = 1:4$. Likelihood ratio of an "against" move: $0.8/0.4 = 2$. Posterior odds $2:4 = 1:2$, so $P = 1/3$.

(b) Three against moves. Each one multiplies the odds by 2:

$$\frac{1}{4} \times 2 \times 2 \times 2 = \frac{8}{4} = 2, \quad \text{odds } 2:1, \quad P = \frac{2}{3}.$$

Count to check: sharp clients with three against moves: $200 \times 0.8^3 = 102.4$. Normal: $800 \times 0.4^3 = 51.2$. Share: $102.4/153.6 = 2/3$.

(c) Need odds of at least $9:1$ (because $9/(9+1) = 90\%$). After n against moves the odds are $2^n/4$. We need $2^n/4 \geq 9$, so $2^n \geq 36$.

- $n = 5$: $2^5 = 32$, odds $8:1$, $P = 8/9 \approx 88.9\%$. Not enough.
- $n = 6$: $2^6 = 64$, odds $16:1$, $P = 16/17 \approx 94.1\%$. Enough.

So **6** in a row.

(d) A move in your favour. For a sharp client, "favour" happens 20% of the time; for a normal client, 60%. Likelihood ratio $0.2/0.6 = 1/3$. It points **away** from sharp.

$$\text{odds} = 2 \times \frac{1}{3} = \frac{2}{3}, \qquad P = \frac{2}{2+3} = \frac{2}{5} = 40\%.$$

Interview trick. Do not redo the whole table at every step. Keep a running odds number and multiply: $\times 2$ for every against move, $\times \frac{1}{3}$ for every move your way. The order of the trades does not matter, only how many of each.

Sanity checks. (a) $1/3$ is above the 20% prior: an against move is evidence of sharpness, so belief should go up, but not to certainty, since normal clients also cause it 40% of the time. (b) More of the same evidence pushes further: $2/3 > 1/3$. (d) One good outcome pulls the belief down from $2/3$ to $2/5$, but not back to the prior, because the earlier evidence still counts. All fractions were checked exactly by computer.

Self-check answers. (1) 1,000 trades: 20 bad, of which 19 flagged; 980 good, of which 98 flagged. Share bad: $19/117 \approx 16.2\%$. (2) $60/190 = 6/19 \approx 31.6\%$. (3) Odds $1:3 \times 6 = 6:3 = 2:1$, probability $2/3$.

---

2. Derivatives: Duration and DV01, or How Hard the Seesaw Swings

> **Where we are.** Lesson 1: compounding, $(1+r)^t$. Lesson 2: discount factors. Lesson 8: a bond's price is the present value of its fixed payments, $P = \sum \text{payment}/(1+y)^t$, and price and yield sit on a seesaw. We also saw that a 1% rise in yields cost the 3-year 4% bond 2.72 points but the 10-year bond 7.72 points. Today we put a single number on "how hard the seesaw swings".

2.1 The scene

A Hong Kong insurer holds USD 500 million of US Treasuries. Its risk manager does not ask "what is the price?" first. She asks: **"how much do we lose if yields go up by one basis point?"** (A basis point, "bp", is one hundredth of a percent: 0.01%.) That number is the **DV01**, "dollar value of one basis point". Every rates trader, every bond fund and every bank treasury knows the DV01 of its book by heart, because it turns "rates moved 7bp today" into "we made or lost this many dollars".

On the other side, a hedge fund that thinks yields will rise wants to be **short** bonds, and needs to know how many bonds to sell to bet exactly USD 50,000 per bp. Same number, different use.

2.2 The central picture: sandbags on a plank

Draw a plank. It is a time line: year 0 at the left end, year 10 at the right. Every payment the bond makes is a **sandbag** placed on the plank at the date it is paid. The weight of each sandbag is the **present value** of that payment (Lesson 2), not its face amount.

For a 10-year 4% bond at a 4% yield, there are nine small bags (the coupons, each worth less than 4 today) and one huge bag at year 10 (the last coupon plus the face value). Now find the point where the plank **balances**. That balance point is the bond's **Macaulay duration**: the average time you wait for your money, weighted by how much each payment is worth.

In the picture	In the bond
The plank	Time, in years
A sandbag at year t	The payment made at year t
Weight of the bag	Present value of that payment, $\mathrm{CF}_t/(1+y)^t$
Total weight	The bond's price P
Balance point	**Macaulay duration** D
Further right = more sensitive	A payment far away is hit harder when yields move

Why does the balance point tell us about risk? Because, as we show in 2.4, a payment due in t years loses **about t times as much** (in percent) as a payment due in 1 year when yields rise a little. Far-away bags shrink most. So a bond whose weight sits far to the right swings hard.

Where the analogy stops: real sandbags do not change weight. Ours do: when yields change, all the present values change, and the balance point shifts a little. Duration is a measurement for **small** moves around today's yield.

2.3 Computing the balance point

Take Lesson 8's **3-year 4% bond** at a yield of 4% (price 100).

Year t	Payment	Present value (weight)	$t\times$ weight
1	4	$4/1.04 = 3.8462$	3.8462
2	4	$4/1.04^2 = 3.6982$	7.3964
3	104	$104/1.04^3 = 92.4556$	277.3669
Total		**100.0000**	**288.6095**

$$D = \frac{\sum_t t \times \mathrm{PV}_t}{P} = \frac{288.6095}{100} = 2.886 \text{ years.}$$

What each symbol means. PV_t is the weight of the bag at year t. $P = \sum_t \mathrm{PV}_t$ is the total weight, the price. The top line adds up "position times weight". Dividing by the total weight gives the balance point, just like finding the average age of a group by "sum of ages divided by number of people".

Read aloud: "duration is the sum of each payment time times that payment's present value, divided by the price."

The answer, 2.886 years, is a bit **less than 3**: the two small coupons pull the balance point slightly to the left. For a **zero-coupon** bond there is only one bag, so the duration equals the maturity exactly.

The same calculation for the **10-year 4% bond** gives $D = 8.435$ years. Far less than 10, because nine coupons pull to the left, but much more than 2.886. That is why it swung almost three times as hard yesterday.

2.4 From balance point to price sensitivity

The idea in words. Nudge the yield up by a tiny amount h (say 1bp $= 0.0001$). Each payment at year t is now divided by $(1+y+h)^t$ instead of $(1+y)^t$. How much smaller does that make it?

Step by step:

1. Factor out: $1 + y + h = (1+y)\left(1 + \dfrac{h}{1+y}\right)$.
2. So $(1+y+h)^t = (1+y)^t \left(1 + \dfrac{h}{1+y}\right)^t$.
3. For a **tiny** rate, compounding t times is almost the same as simple interest (Lesson 1): $\left(1 + \dfrac{h}{1+y}\right)^t \approx 1 + t \cdot \dfrac{h}{1+y}$.
4. Dividing by $(1 + \text{small})$ is almost the same as multiplying by $(1 - \text{small})$. So each payment's value falls by about $t \cdot \dfrac{h}{1+y}$ **as a fraction of itself**.

A bag at year 3 shrinks about three times as much, in percent, as a bag at year 1. Add up all the bags, each shrinking by its own t: the whole price shrinks by the **weighted average** of the t's, which is exactly D. So

$$\frac{\Delta P}{P} \approx -\frac{D}{1+y}\,\Delta y = -D_{\text{mod}}\,\Delta y, \qquad D_{\text{mod}} = \frac{D}{1+y}.$$

What each symbol means. ΔP ("delta P") is the change in price, Δy the change in yield (as a decimal). D_{mod} is the **modified duration**: the percentage price change for a 1.00 change in yield. The minus sign is the seesaw.

Read aloud: “the percentage change in price is about minus the modified duration times the change in yield.”

With numbers. 3-year bond: $D_{\text{mod}} = 2.886/1.04 = 2.775$. For a 1bp rise, the price falls by about $2.775 \times 0.0001 = 0.02775\%$ of its value.

DV01 turns this into money: DV01 $= P \times D_{\text{mod}} \times 0.0001$ (per bp).

- 3-year bond, USD 10 million face at price 100: DV01 $= 10{,}000{,}000 \times 2.775 \times 0.0001 = \text{USD } 2{,}775$.
- 10-year bond, $D_{\text{mod}} = 8.435/1.04 = 8.111$: DV01 $= \text{USD } 8{,}111$.

Check by repricing. Desks also get DV01 by simply re-pricing the bond at $y + 1\text{bp}$ (“bump and reprice”). For the 3-year bond the exact loss is USD 2,774.56: the shortcut was within a dollar.

Where the shortcut drifts. For a big 1% move, duration predicts a loss of USD 277,509 on the 3-year bond; the exact loss is about USD 272,300 (Lesson 8). For the 10-year bond: predicted 811,090, exact 772,170. The real loss is smaller, and for a 1% fall the real gain (853,020) is **bigger** than predicted. That gap is yesterday’s **convexity**: the price curve bends in the holder’s favour. Duration is the straight line that touches the curve; it is excellent for small moves and too pessimistic for big ones.

2.5 Worked trade: hedging a 10-year with a 3-year

A pension fund owns USD 10 million of the 10-year 4% bond (DV01 USD 8,111). The trustees are nervous about rates for the next month but do not want to sell. The fund **shorts** 3-year bonds (sells bonds it borrows, Lesson 3) so that the two DV01s cancel.

Step 1: hedge ratio. Each USD 1 million face of the 3-year has DV01 USD 277.51.

$$\text{Face to short} = \frac{8{,}111}{277.51} \times 1 \text{ million} \approx \text{USD } 29.2 \text{ million}.$$

You need almost three times the face value, because the short bond swings about a third as hard.

Step 2: a parallel 10bp rise (both yields 4.00% to 4.10%):

Position	P&L
Long USD 10m 10-year	$-80{,}707$
Short USD 29.2m 3-year	$+80{,}879$
Net	$+172$

The hedge works: a move that would have cost USD 80,707 now costs almost nothing.

Step 3: but the curve does not always move in parallel. Suppose instead the 10-year yield rises 20bp and the 3-year only 5bp (the curve **steepens**):

Position	P&L
Long 10-year (+20bp)	$-160{,}615$
Short 3-year (+5bp)	$+40{,}478$
Net	$-120{,}137$

A DV01 hedge protects against **parallel** moves only. Different maturities move by different amounts, and that shape risk is tomorrow's topic: the yield curve.

2.6 When duration hit the headlines: UK pensions, September 2022

Many UK pension funds ran "liability-driven investment" (LDI) strategies: they held long-dated gilts (UK government bonds), often with leverage, to match the very long duration of the pensions they owe. On 23 September 2022 the UK government announced a "mini-budget" of unfunded tax cuts. Long gilt yields jumped by more than one percentage point within a few days. With durations of 20 years or more, a 1% move means a price fall of roughly 20% or more. The leveraged funds faced huge collateral calls, had to sell gilts to raise cash, which pushed yields higher still, and the Bank of England stepped in on 28 September 2022 with emergency purchases of long-dated gilts. The lesson: know your DV01, and know what happens when a big move arrives faster than you can post margin (Lesson 7).

2.7 Likely interview questions

Q1. What is the duration of a 5-year zero-coupon bond? Exactly 5 years (Macaulay). All the weight sits on one date. Its modified duration is $5/(1+y)$, a little less.

Q2. Two 10-year bonds, coupons 2% and 8%, both yielding 4%. Which has more duration? The 2% bond. Smaller coupons mean less weight on the left of the plank, so the balance point is further right. At a 4% yield: about 9.07 years versus 7.64 years.

Q3. Your book has DV01 of +USD 50,000 (you gain when yields fall). Yields rise 6bp. Rough P&L? About $-6 \times 50{,}000 = -\text{USD } 300{,}000$. For a much bigger move, convexity would make the true loss a little smaller than the straight-line estimate.

---

3. Algorithms & Data Structures: Recursion and the Call Stack

> **Where we are.** Lesson 1: Big-O, counting steps. Lesson 2: functions, references and loops. Lesson 6: merge sort, which secretly used today's idea: "sort the left half, sort the right half, merge". Today we look at that idea directly: a function that calls itself, and the **call stack** that the computer uses to keep track.

3.1 The central picture: a chain of helpers with sticky notes

You must count the people in a long queue, but you are only allowed to look at the person directly behind you. So you tap that person and ask: "how many people are behind **you**? Tell me, and I will add one." That person does the same thing, and so on. The last person in the queue sees nobody behind them and answers "zero". Then the answers travel back up the queue, each person adding one, until the answer reaches you.

Two things make this work:

1. **A base case.** Someone must answer without asking anyone. Without the last person, the questions would never stop.
2. **A smaller problem each time.** Each helper's queue is one shorter than yours.

While they wait, each person must **remember** what they were doing ("I still have to add one"). The computer does this with a **stack of sticky notes**: every call puts a new note (a **stack frame**) on top, holding its own variables and the place to resume. When a call returns, its note is thrown away and the one below continues. Last in, first out.

In the picture	In the code
Asking the person behind you	A function calling itself with a smaller input
The last person, who answers "zero"	The **base case**
"I will add one when you answer"	Work done **after** the recursive call returns
Sticky note per waiting person	A **stack frame** on the call stack
Pile of notes	The call stack; its height is the **recursion depth**
Queue too long, notes fall over	**Stack overflow**: the program crashes

Where the analogy stops: real people can run off and work in parallel. Our calls run one at a time; the caller is frozen until the helper answers.

3.2 Warm-up: factorial, frame by frame

$n! = n \times (n-1)!$, and $1! = 1$ (Lesson 3 of Probability). In C++:

```
long long factorial(int n) {
    if (n <= 1) return 1;              // base case
    return n * factorial(n - 1);       // trust the helper, add one step
}
```

Calling `factorial(4)`:

Step	Stack (top on the right)	What happens
1	f(4)	needs f(3), waits
2	f(4) f(3)	needs f(2), waits
3	f(4) f(3) f(2)	needs f(1), waits
4	f(4) f(3) f(2) f(1)	base case, returns 1
5	f(4) f(3) f(2)	returns $2 \times 1 = 2$
6	f(4) f(3)	returns $3 \times 2 = 6$
7	f(4)	returns $4 \times 6 = 24$

The pile grows to height 4, then shrinks. **Depth n, time $O(n)$.** The "leap of faith": when you write `factorial(n - 1)`, do not trace it in your head; trust that it returns the right answer for the smaller case, and only check that your one step and your base case are right.

3.3 The problem: fast power

Compute x^n for a real number `x` (a `double`) and an integer `n` (any 32-bit `int`, possibly negative). Do not use `std::pow`. This is the classic interview problem "Pow(x, n)".

- $-100 < x < 100$; n from -2^{31} to $2^{31} - 1$; the answer fits comfortably in a `double`.

x	n	Output
2.0	10	1024
2.1	3	9.261
2.0	-2	0.25 (because $2^{-2} = 1/4$)
1.0	$-2{,}147{,}483{,}648$	1
-1.0	$2{,}147{,}483{,}647$	-1 (odd power)

A quant flavour: $(1 + 0.04/365)^{365} = 1.040808$, the growth of USD 1 at 4% compounded daily for a year (Derivatives Lesson 1).

3.4 Brute force first

Multiply `x` by itself $|n|$ times; if $n < 0$, take one over the result. That is $O(|n|)$ multiplications. For $n = 2{,}147{,}483{,}647$ it is over two billion steps: several seconds. Too slow, and we can do much better.

Writing the brute force **recursively**, as $x^n = x \times x^{n-1}$, would be even worse: depth n means two billion sticky notes. The stack is usually only a few megabytes, enough for tens of thousands of frames, so the program would crash with a stack overflow long before finishing.

3.5 The insight: halve the problem

$x^8 = (x^4)^2$. And $x^4 = (x^2)^2$, and $x^2 = (x^1)^2$. So x^8 takes **3 squarings**, not 7 multiplications.

For odd n, peel off one x: $x^9 = (x^4)^2 \times x$.

So the recursive rule is:

$$x^n = \begin{cases} 1 & n = 0 \quad \text{(base case)} \\ \left(x^{\lfloor n/2 \rfloor}\right)^2 & n \text{ even} \\ \left(x^{\lfloor n/2 \rfloor}\right)^2 \times x & n \text{ odd} \end{cases}$$

($\lfloor n/2 \rfloor$, "floor of n over 2", means $n/2$ rounded down, which is what integer division `n / 2` does in C++ for $n \geq 0$.)

Trace for $n = 13$: $13 \to 6 \to 3 \to 1 \to 0$. Going back up: $x^0 = 1$; $x^1 = 1^2 \cdot x$; $x^3 = (x^1)^2 \cdot x$; $x^6 = (x^3)^2$; $x^{13} = (x^6)^2 \cdot x$. Five calls.

Complexity. Each call halves n, so the number of calls is about $\log_2 n + 1$. For the largest `int`, $n = 2^{31} - 1$, that is **32 calls**: time $O(\log n)$, and the stack is only 32 frames deep, space $O(\log n)$. From two billion steps to 32.

The trap that makes it slow again. If you write `return powRec(x, n/2) * powRec(x, n/2);`, the function calls itself **twice** with the same input. Each call then calls two more, and so on: a tree with about $2n$ calls, back to $O(n)$. Always compute the half **once**, store it in a variable, and square that.

The second trap: negative n. Use $x^{-n} = 1/x^n$. But in C++, an `int` runs from $-2{,}147{,}483{,}648$ to $2{,}147{,}483{,}647$. The number $+2{,}147{,}483{,}648$ does not fit, so `-n` for the smallest `int` overflows (undefined behaviour). Fix: copy `n` into a 64-bit `long long` **before** negating.

3.6 C++ solution

```cpp
#include <climits>

// Fast power for n >= 0: x^n = (x^(n/2))^2, times one extra x if n is odd.
// depth (optional) records how many calls deep the recursion went.
double powRec(double x, long long n, int level, int* depth) {
    if (depth && level > *depth) *depth = level;
    if (n == 0) return 1.0;                          // base case
    double half = powRec(x, n / 2, level + 1, depth); // ONE recursive call, reused twice
    return (n % 2 == 0) ? half * half : half * half * x;
}

// The function an interviewer asks for. n can be any int, including INT_MIN.
double myPow(double x, int n) {
    long long N = n;                  // widen first: -INT_MIN does not fit in an int
    if (N < 0) return 1.0 / powRec(x, -N, 1, nullptr);
    return powRec(x, N, 1, nullptr);
}
```

```cpp
// Bonus: (a^n) mod m with the same recursion, all in exact integer arithmetic.
// Requires m < 2^31 so that a product of two remainders fits in a long long.
long long powMod(long long a, long long n, long long m) {
    if (n == 0) return 1 % m;
    long long half = powMod(a, n / 2, m);
    long long r = half * half % m;
    if (n % 2 == 1) r = r * (a % m) % m;
    return r;
}
```

The full file `2026-11-04.cpp` also has the factorial warm-up, the slow loop as a checker, every example above, and three randomized tests: 100,000 random (x, n) with $|n| \le 300$ against the slow loop; 100,000 with n anywhere in the `int` range against `std::pow` (x very close to 1 so the answer stays finite); and 20,000 exact integer cases of `powMod` against a slow loop. It also confirms the recursion is 32 calls deep for $n = 2^{31} - 1$. All pass.

3.7 Walkthrough of the C++

`double powRec(double x, long long n, ...).` The helper takes `n` as a `long long` (at least 64 bits), so it can hold 2^{31} safely. `x` is passed **by value**: a `double` is 8 bytes, as cheap to copy as a reference, so `const double&` would gain nothing here (unlike yesterday's `const vector<int>&`).

`if (n == 0) return 1.0;`. The base case comes **first**. Every recursive function should start by asking "am I small enough to answer directly?" If the base case were missing or unreachable, the calls would never stop until the stack overflowed.

`double half = powRec(x, n / 2, level + 1, depth);`. One recursive call, result saved in a local variable. Each call gets its **own** `half`, living in its own stack frame; the `half` of an outer call is not touched by inner calls. That is what the sticky notes mean in practice.

`(n % 2 == 0) ? half * half : half * half * x`. The **conditional operator** `cond ? a : b` gives `a` if `cond` is true, else `b`. It is an expression, so it can be returned directly. `n % 2` is the remainder after dividing by 2: 0 for even, 1 for odd (for $n \ge 0$).

`int* depth` and `nullptr`. A **pointer** holds the address of a variable. Passing `&depth` from `main` lets every level of the recursion update the same counter. `nullptr` means "no address": in `myPow` we do not care about depth, so we pass `nullptr`, and `if (depth && ...)` skips the update. The `&&` operator **short-circuits**: if `depth` is null, the right-hand side (`*depth`, which would crash) is never evaluated.

`long long N = n;`. Copying an `int` into a `long long` is always safe (every `int` value fits). After that, `-N` is safe even for `INT_MIN`. Doing `-n` first and then widening would be too late: the overflow already happened in `int`.

`1.0 / powRec(...).` Writing `1.0` (a `double`) rather than `1` makes sure we get floating-point division. With two integers, `1 / 4` in C++ is `0`.

`INT_MIN`, `INT_MAX` **from** `<climits>`**.** The smallest and largest `int`. Good test values for any function that negates or adds to an `int`.

`powMod` **and** `% m` **after every multiply.** Remainders are at most $m - 1 < 2^{31}$, so a product of two is below 2^{62} and fits in a `long long`. Reducing after every step keeps numbers small. This same "square and reduce" idea is how computers compute huge powers for cryptography; in the test file it gives $2^{1{,}000{,}000{,}000} \bmod (10^9 + 7) = 140{,}625{,}001$ in 30 calls.

Floating-point note. The fast version does far fewer multiplications than the loop, so it usually has **less** rounding error, not more. The tests compare with a small relative tolerance rather than `==`, because two correct floating-point methods can differ in the last digit.

Self-check. (1) How many calls does `powRec` make for $n = 1{,}000{,}000$? (2) Why is `factorial(100000)` dangerous even though it is only $O(n)$ time? (3) What goes wrong if the base case is `n == 1` instead of `n == 0`?

Answers. (1) $2^{19} = 524{,}288 \le 10^6 < 2^{20}$, so n has 20 binary digits: 20 calls with $n > 0$, plus one for $n = 0$: 21. (2) Depth 100,000: one frame per call can overflow the stack (and the value overflows `long long` anyway after $20!$). (3) A call with $n = 0$ (for example `myPow(x, 0)`, or $n = 1$ calling $n/2 = 0$) would never stop: $0/2 = 0$ again, forever, until a stack overflow.

---

4. Music: Lynyrd Skynyrd, "Free Bird" (1973)

Context

"Free Bird" closes Lynyrd Skynyrd's first album, **(Pronounced 'Lĕh-'nérd 'Skin-'nérd)**, released in August 1973 and produced by Al Kooper. It was written by guitarist **Allen Collins** and singer **Ronnie Van Zant**. The album version runs about nine minutes. A shortened single edit was released later and reached number 19 in the US, but the long version is the one that became a legend.

The story behind it

The band came from Jacksonville, Florida. Collins had the slow, sad chord sequence for a long time before Van Zant put words to it; the song is a goodbye, from someone who cannot stay in one place, even with a person he cares about. In rehearsals, a roadie named **Billy Powell** played a piano introduction for the song. The band liked it so much that they made him their keyboard player, and his piano opening is on the record.

On stage, the band often dedicated the song to **Duane Allman** of the Allman Brothers Band, who had died in a motorcycle crash in 1971, and it became their show closer. The 1976 live album **One More from the Road** has a famous version of around fourteen

minutes. In October 1977, a plane crash killed Ronnie Van Zant and other members of the band and crew, and after that the song took on a new meaning as a farewell.

Why people love it

It is two songs in one. The first half is a slow, gentle ballad with piano and a weeping slide guitar. Then the tempo kicks up and it becomes one of the longest and most celebrated guitar solos in rock, several minutes of guitars trading lines over a driving beat. The emotional build, from quiet regret to pure release, is what makes crowds lose their minds. Shouting “Free Bird!” at concerts of any band has become a running joke because the song is so loved.

What to listen for

- **The piano opening** by Billy Powell, which sets the sad, reflective mood before the voice comes in.
- **The slide guitar** (played by Gary Rossington) in the slow section: a glass or metal slide on the strings makes the notes glide and cry like a voice.
- **The gear change.** Listen for the moment the drums pick up speed and the song shifts from ballad to jam. Notice how the band builds tension for a few bars before it fully takes off.
- **The guitar solo**, led by Allen Collins: it does not try to be clever; it repeats and builds simple ideas faster and higher until the final crash. Count how long the band keeps the energy rising without the song ever getting boring.